SMARTCAREER • STEP-BY-STEP ASSESSMENT QA MANUAL

คู่มือปฏิบัติการทดสอบคลังข้อสอบมาตรฐานแบบจับมือทำ (ครบทั้ง 13 ชุดข้อสอบ)

เอกสารคู่มือสำหรับทดสอบข้อสอบปฏิบัติ (Practical Coding) และข้อสอบทฤษฎี (Theory) ที่อ้างอิงหลักสูตรจาก **freeCodeCamp.org**, **MDN Web Docs** และ **W3Schools.com** พร้อมโค้ดเฉลยสำหรับคัดลอกไปวางและผลลัพธ์ที่ต้องแสดงในแต่ละขั้นตอน

ผู้ใช้งานเป้าหมาย: ทีม QA / ผู้ดูแลระบบ (Admin) / ผู้สมัครงาน (Candidate)

จำนวนข้อสอบทั้งหมด: 13 ชุดข้อสอบ (31 ข้อย่อย • 100+ ชุดทดสอบย่อย)

🚀 ขั้นตอนภาพรวมตั้งแต่เริ่มต้นเข้าสู่ระบบจนจบการทดสอบ (6 ขั้นตอนหลัก)

1 เข้าสู่ระบบ Candidate (/login)
ล็อกอินด้วยบัญชีผู้สมัครงาน (GitHub / Demo Candidate) แล้วดูค่าเริ่มต้นบนกราฟเรดาร์ที่หน้า `/profile`

2 เปิดศูนย์สอบ (/assessments)
ตรวจสอบรายการข้อสอบทั้ง 13 ชุด แล้วคลิกปุ่ม **“เริ่มทำแบบทดสอบ →”** ในชุดที่ต้องการ

3 รันโค้ดตั้งต้น (Negative Test)
กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ด้วย Starter Code เดิม เพื่อดูข้อความแจ้งเตือนสีแดง `FAILED`

4 วางโค้ดเฉลย & กดรันทดสอบ
คัดลอกโค้ดเฉลยจากคู่มือนี้ไปวางใน Code Editor กดรันทดสอบเพื่อดูผล `PASSED` สีเขียวครบทุกข้อตัวอย่าง

5 บันทึกคำตอบรายข้อ & Finalize
กด **“บันทึกและส่งคำตอบข้อนี้”** จนขึ้นเครื่องหมาย ✓ ครบทุกข้อ แล้วกด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**

6 ตรวจเหรียญรางวัล & กราฟเรดาร์
ดูคะแนนสุทธิ `100% PASSED`, เหรียญ Verified Badge, ปุ่มดูเฉลยรายข้อ และกราฟเรดาร์ 5 แกนในหน้า `/profile`

🎯 เช็กลิสต์ตรวจสอบความถูกต้องระดับระบบหลังทำข้อสอบครบ (End-to-End Verification Checklist)

1. **ตรวจสอบเหรียญรับรองทักษะ (Verified Skill Badges) ที่หน้า** `/assessments` : กล่องสีทองด้านบนสุดต้องแสดงเหรียญ `SmartCareer Verified` ของทุกทักษะที่สอบผ่านเกณฑ์ (เฉพาะข้อสอบ `PRACTICAL_CODING`)
2. **ตรวจสอบหน้าต่างวิเคราะห์ผลรายข้อ (Attempt Review Modal)**: ในตาราง *“ประวัติการเข้าสอบของฉัน”* คลิกปุ่ม **“ดูผลวิเคราะห์ / เฉลย”** ต้องเปิดหน้าต่างแสดงคะแนนสุทธิ, คะแนนดิบ, โค้ดที่ส่ง และคำอธิบายแนวคิดที่บันทึกไว้
3. **ตรวจสอบกราฟเรดาร์ทักษะ 5 แกน ที่หน้า** `/profile` : เมื่อสอบผ่านครบทั้ง 5 หมวด (`Frontend`, `Backend`, `Database`, `DevOps`, `Testing`) กราฟเรดาร์ทั้ง 5 แกนต้องแสดงคะแนน **100%** ครบทุกแกน และตารางทักษะด้านล่างแสดงป้ายสถานะสีเขียว `ASSESSED`
4. **ตรวจสอบการคำนวณคะแนนจับคู่งาน (Job Matching Score) ที่หน้า** `/jobs` : คลิกเข้าไปดูรายละเอียดงาน เช่น *Full Stack JavaScript Developer* หรือ *Frontend Engineer (React / TypeScript)* สถานะความพร้อมจะเปลี่ยนเป็น **ASSESSED (ความครอบคลุม 100%)** พร้อมคะแนนจับคู่งานเต็ม **100%**

ตารางสรุปคลังข้อสอบมาตรฐานทั้งหมดในระบบ (13 ชุดข้อสอบ)

ลำดับ	ชื่อชุดข้อสอบ (Assessment Title & Slug)	แหล่งอ้างอิง	ทักษะ (หมวดหมู่)	ประเภท	เวลา / เกณฑ์ผ่าน	จำนวน
ชุดที่ 1	**JavaScript Algorithms & Data Structures (freeCodeCamp Reference)** `fcc-js-algorithms-data-structures-v1`	freeCodeCamp.org	**JavaScript** (FRONTEND)	PRACTICAL_CODING	45 นาที / ผ่าน 75%	3 ข้อ
ชุดที่ 2	**Node.js Backend & REST API Engineering (freeCodeCamp Reference)** `fcc-nodejs-backend-api-v1`	freeCodeCamp.org	**Node.js** (BACKEND)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 3	**Automated Testing & Quality Assurance (freeCodeCamp Reference)** `fcc-automated-testing-qa-v1`	freeCodeCamp.org	**Automated Testing** (TESTING)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 4	**TypeScript Type-Safe Data Modeling (MDN Web Docs Reference)** `mdn-typescript-data-modeling-v1`	MDN Web Docs	**TypeScript** (FRONTEND)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 5	**React State Management & UI Selectors (MDN Web Docs Reference)** `mdn-react-state-architecture-v1`	MDN Web Docs	**React** (FRONTEND)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 6	**Web Security, HTTP & Browser Standards (MDN Web Docs Reference)** `mdn-web-security-http-theory-v1`	MDN Web Docs	**Application Security** (SECURITY)	THEORY	25 นาที / ผ่าน 75%	5 ข้อ
ชุดที่ 7	**PostgreSQL & Relational Query Logic (W3Schools Reference)** `w3s-postgresql-relational-data-v1`	W3Schools.com	**PostgreSQL** (DATABASE)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 8	**Python Data Processing & Analytics (W3Schools Reference)** `w3s-python-data-analytics-v1`	W3Schools.com	**Python** (BACKEND)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 9	**Docker & DevOps Configuration Linter (W3Schools Reference)** `w3s-docker-devops-config-v1`	W3Schools.com	**Docker** (DEVOPS)	PRACTICAL_CODING	45 นาที / ผ่าน 70%	3 ข้อ
ชุดที่ 10	**JavaScript Practical Coding Assessment** `javascript-practical-coding-v1`	SmartCareer Core	**JavaScript** (FRONTEND)	PRACTICAL_CODING	30 นาที / ผ่าน 70%	2 ข้อ
ชุดที่ 11	**SQL & Data Query Logic Assessment** `sql-data-query-practical-v1`	SmartCareer Core	**SQL** (DATABASE)	PRACTICAL_CODING	30 นาที / ผ่าน 60%	1 ข้อ
ชุดที่ 12	**Python Practical Problem Solving** `python-practical-coding-v1`	SmartCareer Core	**Python** (BACKEND)	PRACTICAL_CODING	30 นาที / ผ่าน 70%	1 ข้อ
ชุดที่ 13	**Web & Cloud Engineering Fundamentals (Theory)** `web-engineering-theory-v1`	SmartCareer Core	**JavaScript** (FRONTEND)	THEORY	20 นาที / ผ่าน 70%	2 ข้อ

หมวด A: คลังข้อสอบอ้างอิงจาก freeCodeCamp.org (3 ชุดข้อสอบ)

อ้างอิงหลักสูตร JavaScript Algorithms & Data Structures, Back End Development & APIs และ Quality Assurance

ชุดข้อสอบที่ 1 freeCodeCamp.org สอบปฏิบัติ (อัปเดต Global Skill Score)

⏱ เวลา 45 นาที • เกณฑ์ผ่าน 75% • ทักษะ: **JavaScript (FRONTEND)**

JavaScript Algorithms & Data Structures (freeCodeCamp Reference)

อ้างอิงหลักสูตร JavaScript Algorithms and Data Structures จาก freeCodeCamp.org วัดทักษะการจัดการสตริง การแปลงระบบตัวเลข และอัลกอริทึมเชิงตรรกะระดับใช้งานจริง

ชุดที่ 1 • ข้อที่ 1/3 Alphanumeric Palindrome Validator

EASY | 30 คะแนน | JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

อ้างอิงโจทย์ Palindrome Checker จาก freeCodeCamp: เขียนฟังก์ชัน `solution(text)` ที่ตรวจสอบว่าสตริง `text` เป็น Palindrome หรือไม่ โดยตัดอักขระที่ไม่ใช่ตัวอักษรภาษาอังกฤษหรือตัวเลข (non-alphanumeric) และช่องว่างออกทั้งหมด และไม่สนใจตัวพิมพ์เล็ก-ใหญ่ คืนค่าเป็น `true` หรือ `false`

🛠 **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(text) {
  const cleaned = String(text || '').toLowerCase().replace(/[^a-z0-9]/g, '');
  const reversed = cleaned.split('').reverse().join('');
  return cleaned === reversed;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้ Regular Expression /[^a-z0-9]/g ตัดอักขระพิเศษออก แปลงเป็นตัวพิมพ์เล็ก แล้วเปรียบเทียบกับสตริงที่กลับด้าน`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **5 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 3 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`["A man, a plan, a canal. Panama"]`	`true`
#2	Visible Test Case	`["race a car"]`	`false`
#3	Hidden Test Case	`["_eye"]`	`true`
#4	Hidden Test Case	`["0_0 (: /-\\ :) 0-0"]`	`true`
#5	Hidden Test Case	`["almostomla"]`	`false`

ชุดที่ 1 • ข้อที่ 2/3 **Roman Numeral Converter** MEDIUM 35 คะแนน JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

อ้างอิงโจทย์ Roman Numeral Converter จาก freeCodeCamp: เขียนฟังก์ชัน `solution(num)` ที่แปลงตัวเลขจำนวนเต็มบวก `num` (1 ถึง 3999) ให้เป็นสตริงเลขโรมันมาตรฐาน (เช่น 1994 -> `"MCMXCIV"`) หากอยู่นอกช่วงให้คืนค่า `""`

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(num) {
  const n = Number(num);
  if (!Number.isInteger(n) || n < 1 || n > 3999) return '';
  const pairs = [
    [1000, 'M'], [900, 'CM'], [500, 'D'], [400, 'CD'],
    [100, 'C'], [90, 'XC'], [50, 'L'], [40, 'XL'],
    [10, 'X'], [9, 'IX'], [5, 'V'], [4, 'IV'], [1, 'I']
  ];
  let rem = n;
  let out = '';
  for (const [val, sym] of pairs) {
    while (rem >= val) {
      out += sym;
      rem -= val;
    }
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `จับคู่ค่าตัวเลขกับสัญลักษณ์โรมันจากมากไปน้อย (รวมรูปย่อ 900, 400, 90, 40, 9, 4) แล้วหักลบค่าไปเรื่อยๆ แบบ Greedy`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **5 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 3 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[36]`	`XXXVI`
#2	👁 Visible Test Case	`[1994]`	`MCMXCIV`
#3	🔒 Hidden Test Case	`[3999]`	`MMMCMXCIX`
#4	🔒 Hidden Test Case	`[44]`	`XLIV`
#5	🔒 Hidden Test Case	`[58]`	`LVIII`

ชุดที่ 1 • ข้อที่ 3/3 Caesar Cipher ROT13 Decoder

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

อ้างอิงโจทย์ Caesars Cipher (ROT13) จาก freeCodeCamp: เขียนฟังก์ชัน `solution(encoded)` ที่ถอดรหัสสตริงตัวพิมพ์ใหญ่โดยเลื่อนตัวอักษรภาษาอังกฤษ A-Z ไปข้างหน้า 13 ตำแหน่ง (เช่น `"SERR PBQR PNZC"` -> `"FREE CODE CAMP"`) โดยคงช่องว่างและเครื่องหมายวรรคตอนไว้ตามเดิม

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(encoded) {
  return String(encoded || '').replace(/[A-Z]/g, (ch) => {
    const code = ch.charCodeAt(0) - 65;
    return String.fromCharCode(((code + 13) % 26) + 65);
  });
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `แปลงรหัส ASCII ของ A-Z (65-90) ด้วยสูตร ``((code - 65 + 13) % 26) + 65`` โดยไม่แตะต้องอักขระที่ไม่ใช่ A-Z`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥75%)** ได้รับเหรียญ **SmartCareer Verified — JavaScript** และอัปเดตคะแนนแกน **FRONTEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`["SERR PBQR PNZC"]`	`FREE CODE CAMP`
#2	Visible Test Case	`["SERR CVMMN!"]`	`FREE PIZZA!`
#3	🔒 Hidden Test Case	`["GUR DHVPX OEBJA SBK WHZCF BIRE GUR YNML QBT."]`	`THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG.`
#4	🔒 Hidden Test Case	`["123-ABC"]`	`123-NOP`

ชุดข้อสอบที่ 2 | freeCodeCamp.org | สอบปฏิบัติ (อัปเดต Global Skill Score) ⏱ เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **Node.js (BACKEND)**

Node.js Backend & REST API Engineering (freeCodeCamp Reference)

อ้างอิงหลักสูตร Back End Development and APIs จาก freeCodeCamp.org วัดทักษะการจัดการ Pagination, Rate Limiting และการประมวลผล Webhook แบบ Idempotent สำหรับงานวิศวกร Backend

ชุดที่ 2 • ข้อที่ 1/3 Sanitize REST Pagination Parameters

EASY | 30 คะแนน | JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

อ้างอิงมาตรฐาน REST API จาก freeCodeCamp Back End Development: เขียนฟังก์ชัน `solution(query)` ที่รับอ็อบเจกต์ `{ page, limit, totalItems }` แล้วคืนค่า `{ page, limit, offset, totalPages }` โดยมีเกณฑ์ดังนี้:
- `page` ต้องเป็นจำนวนเต็มตั้งแต่ 1 ขึ้นไป (ค่าเริ่มต้น 1)
- `limit` ต้องเป็นจำนวนเต็มระหว่าง 1 ถึง 100 (ค่าเริ่มต้น 20, หากเกิน 100 ให้ปัดเป็น 100, หากน้อยกว่า 1 ให้ใช้ 20)
- `offset = (page - 1) * limit`
- `totalPages = Math.max(1, Math.ceil(totalItems / limit))`

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(query) {
  const q = query || {};
  const rawPage = parseInt(q.page, 10);
  const page = Number.isFinite(rawPage) && rawPage >= 1 ? rawPage : 1;
  const rawLimit = parseInt(q.limit, 10);
  let limit = 20;
  if (Number.isFinite(rawLimit) && rawLimit >= 1) {
    limit = Math.min(100, rawLimit);
  }
  const totalItems = Math.max(0, Number(q.totalItems) || 0);
  const offset = (page - 1) * limit;
  const totalPages = Math.max(1, Math.ceil(totalItems / limit));
  return { page, limit, offset, totalPages };
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `แปลงค่า page และ limit ด้วย parseInt พร้อมทำ Boundary Clamping เพื่อป้องกันการดึงข้อมูลเกินพิกัดใน REST API`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[{"page": 2, "limit": 10, "totalItems": 45}]`	`{"page":2,"limit":10,"offset":10,"totalPages":5}`
#2	Visible Test Case	`[{"page": -1, "limit": 500, "totalItems": 250}]`	`{"page":1,"limit":100,"offset":0,"totalPages":3}`
#3	Hidden Test Case	`[{"totalItems": 0}]`	`{"page":1,"limit":20,"offset":0,"totalPages":1}`
#4	Hidden Test Case	`[{"page": "3", "limit": "0", "totalItems": 60}]`	`{"page":3,"limit":20,"offset":40,"totalPages":3}`

ชุดที่ 2 • ข้อที่ 2/3 Sliding Window API Rate Limiter

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(timestamps, windowMs, maxRequests)` ที่จำลองการทำงานของ Middleware Rate Limiter สำหรับ Node.js API โดยรับอาร์เรย์ของเวลาที่เกิด Request (`timestamps` เรียงจากน้อยไปมาก), ขนาดหน้าต่างเวลา `windowMs` , และโควตาสูงสุด `maxRequests` ในช่วง `[t - windowMs + 1, t]` คืนค่าอาร์เรย์ของ `boolean` (`true` ถ้าอนุญาต, `false` ถ้าถูกบล็อกและไม่นับรวมในโควตา)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(timestamps, windowMs, maxRequests) {
  const accepted = [];
  const out = [];
  for (const t of timestamps || []) {
    while (accepted.length > 0 && accepted[0] <= t - windowMs) {
      accepted.shift();
    }
    if (accepted.length < maxRequests) {
      accepted.push(t);
      out.push(true);
    } else {
      out.push(false);
    }
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้คิวเก็บเฉพาะ Timestamp ของ Request ที่ได้รับอนุญาตภายในหน้าต่างเวลา `[t - windowMs + 1, t]``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[[100, 200, 300, 1100], 1000, 2]`	`[true,true,false,true]`
#2	👁 Visible Test Case	`[[10, 20, 30, 40], 100, 3]`	`[true,true,true,false]`
#3	🔒 Hidden Test Case	`[[1000, 2000, 3000], 500, 1]`	`[true,true,true]`
#4	🔒 Hidden Test Case	`[[], 1000, 5]`	`[]`

ชุดที่ 2 • ข้อที่ 3/3 Idempotent Webhook Ledger Aggregator

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(events)` ที่รับรายการ Webhook Events รูปแบบ `{ eventId, accountId, amount }` โดยระบบอาจส่ง Event ซ้ำ (`eventId` เดิม) มาได้ ให้ประมวลผลแต่ละ `eventId` เพียงครั้งแรกเท่านั้น และคืนค่าอาร์เรย์ของ `{ accountId, balance }` เรียงลำดับตาม `accountId` (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(events) {
  const seen = new Set();
  const balances = new Map();
  for (const ev of events || []) {
    if (!ev || !ev.eventId || !ev.accountId) continue;
    if (seen.has(ev.eventId)) continue;
    seen.add(ev.eventId);
    const prev = balances.get(ev.accountId) || 0;
    balances.set(ev.accountId, prev + Number(ev.amount || 0));
  }
  return Array.from(balances.entries())
    .sort((a, b) => a[0].localeCompare(b[0]))
    .map(([accountId, balance]) => ({ accountId, balance }));
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้ `Set` เก็บ `eventId` ที่เคยประมวลผลแล้วเพื่อรับประกัน Idempotency และใช้ `Map` รวมยอดตาม `accountId``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — Node.js** และอัปเดตคะแนนแกน **BACKEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"eventId":"e1","accountId":"acc-b","amount":100},{"eventId":"e1","accountId":"acc-b","amount":100},{"eventId":"e2","accountId":"acc-a","amount":50}]]`	`[{"accountId":"acc-a","balance":50},{"accountId":"acc-b","balance":100}]`
#2	Visible Test Case	`[[{"eventId":"x1","accountId":"u1","amount":200},{"eventId":"x2","accountId":"u1","amount":-50}]]`	`[{"accountId":"u1","balance":150}]`
#3	Hidden Test Case	`[[{"eventId":"dup","accountId":"z","amount":500},{"eventId":"dup","accountId":"z","amount":999}]]`	`[{"accountId":"z","balance":500}]`
#4	Hidden Test Case	`[[]]`	`[]`

ชุดข้อสอบที่ 3 freeCodeCamp.org สอบปฏิบัติ (อัปเดต Global Skill Score)

⏱ เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **Automated Testing (TESTING)**

Automated Testing & Quality Assurance (freeCodeCamp Reference)

อ้างอิงหลักสูตร Quality Assurance จาก freeCodeCamp.org วัดทักษะการออกแบบ Boundary Value Analysis, การตรวจสอบ API Contract Schema และการวิเคราะห์ Flaky Tests ในระบบ CI/CD

ชุดที่ 3 • ข้อที่ 1/3 Boundary Value Partition Classifier

EASY 30 คะแนน JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

ในการทดสอบแบบ Boundary Value Analysis (BVA) ให้เขียนฟังก์ชัน `solution(values, min, max)` ที่รับอาร์เรย์ของตัวเลข `values` และจำแนกแต่ละค่าเป็น:
- `"OUT_OF_RANGE"` หาก `< min` หรือ `> max`
- `"BOUNDARY"` หากเท่ากับ `min`, `min + 1`, `max - 1`, หรือ `max`
- `"NOMINAL"` หากอยู่ภายในช่วงปกติที่ไม่ใช่ขอบเขต
คืนค่าเป็นอาร์เรย์ของสตริงสถานะตามลำดับ

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(values, min, max) {
  return (values || []).map((v) => {
    if (typeof v !== 'number' || v < min || v > max) return 'OUT_OF_RANGE';
    if (v === min || v === min + 1 || v === max - 1 || v === max) return 'BOUNDARY';
    return 'NOMINAL';
  });
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ตรวจสอบเงื่อนไขนอกช่วงก่อน จากนั้นตรวจค่าขอบเขต `min, min+1, max-1, max` และค่าทั่วไปเป็น `NOMINAL``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[[0, 1, 2, 5, 9, 10, 11], 1, 10]`	`["OUT_OF_RANGE","BOUNDARY","BOUNDARY","NOMINAL","BOUNDARY","BOUNDARY","OUT_OF_RANGE"]`
#2	👁 Visible Test Case	`[[-1, 0, 50, 100], 0, 100]`	`["OUT_OF_RANGE","BOUNDARY","NOMINAL","BOUNDARY"]`
#3	🔒 Hidden Test Case	`[[10, 11], 10, 11]`	`["BOUNDARY","BOUNDARY"]`
#4	🔒 Hidden Test Case	`[[], 1, 5]`	`[]`

ชุดที่ 3 • ข้อที่ 2/3 API Response Contract Validator

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(payload, schema)` เพื่อตรวจสอบผลลัพธ์จาก API ในชุดทดสอบ Integration Test โดย `schema` เป็นอ็อบเจกต์ที่ระบุชนิดข้อมูลที่ต้องการ เช่น `{ id: "string", score: "number", active: "boolean", tags: "array" }` ให้คืนค่าอาร์เรย์ของชื่อฟิลด์ที่หายไปหรือมีชนิดข้อมูลไม่ตรงกับ `schema` เรียงตามตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(payload, schema) {
  const obj = payload && typeof payload === 'object' ? payload : {};
  const rules = schema && typeof schema === 'object' ? schema : {};
  const invalid = [];
  for (const [key, expectedType] of Object.entries(rules)) {
    const val = obj[key];
    if (expectedType === 'array') {
      if (!Array.isArray(val)) invalid.push(key);
    } else if (expectedType === 'number') {
      if (typeof val !== 'number' || Number.isNaN(val)) invalid.push(key);
    } else if (typeof val !== expectedType) {
      invalid.push(key);
    }
  }
  return invalid.sort();
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `วนลูปตรวจแต่ละฟิลด์ใน `schema` โดยแยกกรณี `array` ด้วย `Array.isArray` และตรวจ `typeof` สำหรับชนิดพื้นฐาน`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[{"id":"u1","score":95,"tags":["js"]},{"id":"string","score":"number","tags":"array"}]`	`[]`
#2	Visible Test Case	`[{"id":123,"score":"95","tags":{}},{"id":"string","score":"number","tags":"array"}]`	`["id","score","tags"]`
#3	Hidden Test Case	`[{"active":true},{"active":"boolean","missingField":"string"}]`	`["missingField"]`
#4	Hidden Test Case	`[{}, {"b":"number","a":"string"}]`	`["a","b"]`

ชุดที่ 3 • ข้อที่ 3/3 CI Flaky Test Detector

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(runs)` ที่รับอาร์เรย์ของผลการรันชุดทดสอบ `{ testName, passed }` ใน CI Pipeline แล้วค้นหาชื่อการทดสอบที่เป็น **Flaky Test** (คือการทดสอบชื่อเดียวกันที่มีทั้งผล `passed: true` และ `passed: false` ปะปนกัน) คืนค่าเป็นอาร์เรย์ของชื่อ Flaky Test เรียงตามตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(runs) {
  const stats = new Map();
  for (const r of runs || []) {
    if (!r || !r.testName) continue;
    const cur = stats.get(r.testName) || { pass: false, fail: false };
    if (r.passed) cur.pass = true;
    else cur.fail = true;
    stats.set(r.testName, cur);
  }
  return Array.from(stats.entries())
    .filter(([, s]) => s.pass && s.fail)
    .map(([name]) => name)
    .sort();
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `เก็บสถานะว่าแต่ละ \`testName\` เคยผ่าน (\`pass\`) และเคยตก (\`fail\`) หรือไม่ หากมีทั้งสองสถานะถือเป็น Flaky Test`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — Automated Testing** และอัปเดตคะแนนแกน **TESTING** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[[{"testName":"auth.login","passed":true},{"testName":"auth.login","passed":false},{"testName":"db.connect","passed":true},{"testName":"db.connect","passed":true}]]`	`["auth.login"]`
#2	👁 Visible Test Case	`[[{"testName":"z.spec","passed":false},{"testName":"a.spec","passed":true},{"testName":"z.spec","passed":true},{"testName":"a.spec","passed":false}]]`	`["a.spec","z.spec"]`
#3	🔒 Hidden Test Case	`[[{"testName":"always.fail","passed":false},{"testName":"always.fail","passed":false}]]`	`[]`
#4	🔒 Hidden Test Case	`[[]]`	`[]`

หมวด B: คลังข้อสอบอ้างอิงจาก MDN Web Docs — Mozilla Developer Network (3 ชุดข้อสอบ)

อ้างอิงหลักสูตร MDN Curriculum ด้าน TypeScript, React State Management และมาตรฐานความปลอดภัยเว็บ (Web Security & HTTP)

ชุดข้อสอบที่ 4 | MDN Web Docs | สอบปฏิบัติ (อัปเดต Global Skill Score) | เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **TypeScript (FRONTEND)**

TypeScript Type-Safe Data Modeling (MDN Web Docs Reference)

อ้างอิงหลักสูตร MDN Web Docs Curriculum ด้าน TypeScript และ Modern JavaScript วัดทักษะการเขียน Runtime Type Guards, Discriminated Union State Reducers และการจัดการข้อมูลแบบ Immutable

ชุดที่ 4 • ข้อที่ 1/3 Runtime Type Guard Validator

EASY | 30 คะแนน | JavaScript (Node.js)

โจทย์คำถาม (Problem Prompt):

อ้างอิงแนวคิด Type Narrowing & Type Guards จาก MDN / TypeScript Handbook: เขียนฟังก์ชัน `solution(input)` เพื่อตรวจสอบว่าข้อมูล `input` มีโครงสร้างตรงตามอินเทอร์เฟซต่อไปนี้หรือไม่:
- `id`: สตริงที่ไม่ใช่ค่าว่างหลังตัดช่องว่าง
- `email`: สตริงที่มีเครื่องหมาย `@`
- `roles`: อาร์เรย์ที่มีสมาชิกอย่างน้อย 1 ตัว และทุกตัวต้องเป็นสตริง
คืนค่า `true` หรือ `false`

ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(input) {
  if (!input || typeof input !== 'object' || Array.isArray(input)) return false;
  if (typeof input.id !== 'string' || input.id.trim().length === 0) return false;
  if (typeof input.email !== 'string' || !input.email.includes('@')) return false;
  if (!Array.isArray(input.roles) || input.roles.length === 0) return false;
  return input.roles.every((r) => typeof r === 'string' && r.trim().length > 0);
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ตรวจสอบชนิดข้อมูลของอ็อบเจกต์และพร็อพเพอร์ตี้แต่ละตัวแบบ Strict Runtime Type Guard ก่อนนำไปใช้งาน`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[{"id":"usr-1","email":"dev@mozilla.org","roles":["CANDIDATE"]}]`	`true`
#2	Visible Test Case	`[{"id":"","email":"dev@mozilla.org","roles":["CANDIDATE"]}]`	`false`
#3	Hidden Test Case	`[{"id":"usr-2","email":"invalid-email","roles":["ADMIN"]}]`	`false`
#4	Hidden Test Case	`[{"id":"usr-3","email":"a@b.com","roles":[123]}]`	`false`

ชุดที่ 4 • ข้อที่ 2/3 Discriminated Union Async State Machine

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(actions)` ที่จำลองการเปลี่ยนสถานะของ Async State Machine จากลำดับ Action แบบ Discriminated Union:
- เริ่มต้นที่ `{ status: "IDLE", data: null, error: null, attempts: 0 }`
- `type: "FETCH_START"` -> `status: "LOADING", error: null, attempts: attempts + 1`
- `type: "FETCH_SUCCESS"` -> `status: "SUCCESS", data: action.payload, error: null`
- `type: "FETCH_ERROR"` -> `status: "ERROR", data: null, error: action.message`
- `type: "RESET"` -> กลับสู่ค่าเริ่มต้นทั้งหมด

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(actions) {
  let state = { status: 'IDLE', data: null, error: null, attempts: 0 };
  for (const act of actions || []) {
    if (!act || !act.type) continue;
    if (act.type === 'FETCH_START') {
      state = { ...state, status: 'LOADING', error: null, attempts: state.attempts + 1 };
    } else if (act.type === 'FETCH_SUCCESS') {
      state = { ...state, status: 'SUCCESS', data: act.payload ?? null, error: null };
    } else if (act.type === 'FETCH_ERROR') {
      state = { ...state, status: 'ERROR', data: null, error: act.message ?? 'Unknown error' };
    } else if (act.type === 'RESET') {
      state = { status: 'IDLE', data: null, error: null, attempts: 0 };
    }
  }
  return state;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้รูปแบบ Discriminated Union แยกตามพร็อพเพอร์ตี้ `type` และอัปเดตสถานะอย่างเป็นลำดับโดยไม่กลายพันธุ์ค่าเดิม`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"type":"FETCH_START"},{"type":"FETCH_SUCCESS","payload":{"items":3}}]]`	`{"status":"SUCCESS","data":{"items":3},"error":null,"attempts":1}`
#2	Visible Test Case	`[[{"type":"FETCH_START"},{"type":"FETCH_ERROR","message":"Timeout"},{"type":"FETCH_START"}]]`	`{"status":"LOADING","data":null,"error":null,"attempts":2}`
#3	Hidden Test Case	`[[{"type":"FETCH_START"},{"type":"RESET"}]]`	`{"status":"IDLE","data":null,"error":null,"attempts":0}`
#4	Hidden Test Case	`[[]]`	`{"status":"IDLE","data":null,"error":null,"attempts":0}`

ชุดที่ 4 • ข้อที่ 3/3 Generic GroupBy & Summary Mapper

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(records, keyField, valueField)` ที่จัดกลุ่มอาร์เรย์ของอ็อบเจกต์ `records` ตามค่าของ `records[i][keyField]` และรวมผลบวกของตัวเลขใน `records[i][valueField]` คืนค่าเป็นอ็อบเจกต์ที่เรียงลำดับคีย์ตามตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(records, keyField, valueField) {
  const map = new Map();
  for (const item of records || []) {
    if (!item || item[keyField] === undefined || item[keyField] === null) continue;
    const k = String(item[keyField]);
    const v = Number(item[valueField]) || 0;
    map.set(k, (map.get(k) || 0) + v);
  }
  const out = {};
  for (const k of Array.from(map.keys()).sort()) {
    out[k] = map.get(k);
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ดึงค่าคีย์และค่าตัวเลขแบบ Dynamic Property Access แล้วจัดเรียงคีย์ก่อนสร้างอ็อบเจกต์ผลลัพธ์`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — TypeScript** และอัปเดตคะแนนแกน **FRONTEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"category":"FRONTEND","points":40},{"category":"BACKEND","points":50},{"category":"FRONTEND","points":35}], "category", "points"]`	`{"BACKEND":50,"FRONTEND":75}`
#2	Visible Test Case	`[[{"team":"B","hours":5},{"team":"A","hours":8}], "team", "hours"]`	`{"A":8,"B":5}`
#3	🔒 Hidden Test Case	`[[], "category", "points"]`	`{}`

ชุดข้อสอบที่ 5 | MDN Web Docs | สอบปฏิบัติ (อัปเดต Global Skill Score) | เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **React (FRONTEND)**

React State Management & UI Selectors (MDN Web Docs Reference)

อ้างอิงหลักสูตร Front-End Frameworks & React จาก MDN Web Docs วัดทักษะการจัดการ Immutable State, การคำนวณ Derived UI State และการแปลงโครงสร้างข้อมูลแบบ Tree สำหรับ Component

ชุดที่ 5 • ข้อที่ 1/3 Immutable Cart State Reducer

EASY | 30 คะแนน | JavaScript (Node.js)

โจทย์คำถาม (Problem Prompt):

อ้างอิงแนวคิด React `useReducer` จาก MDN / React Docs: เขียนฟังก์ชัน `solution(initialItems, action)` ที่อัปเดตรายการสินค้า `[{ id, qty }]` แบบ Immutable:
- `action.type === "ADD"`: หากมี `id` อยู่แล้วให้บวก `qty` เพิ่มตาม `action.qty` หากยังไม่มีให้ต่อท้ายรายการใหม่
- `action.type === "REMOVE"`: ลบรายการที่มี `id` ตรงกับ `action.id`
- หากรายการใดมี `qty <= 0` ให้ตัดออกจากผลลัพธ์เสมอ

ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(initialItems, action) {
  const items = Array.isArray(initialItems) ? initialItems.map((x) => ({ ...x })) : [];
  if (!action || !action.type) return items.filter((x) => x.qty > 0);
  if (action.type === 'REMOVE') {
    return items.filter((x) => x.id !== action.id && x.qty > 0);
  }
  if (action.type === 'ADD') {
    const idx = items.findIndex((x) => x.id === action.id);
    if (idx >= 0) {
      items[idx].qty += Number(action.qty || 0);
    } else if (Number(action.qty || 0) > 0) {
      items.push({ id: action.id, qty: Number(action.qty) });
    }
  }
  return items.filter((x) => x.qty > 0);
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `สร้างสำเนาอาร์เรย์ใหม่เสมอเพื่อให้ React ตรวจจับการเปลี่ยนแปลง Reference ได้อย่างถูกต้อง`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"id":"p1","qty":1}], {"type":"ADD","id":"p1","qty":2}]`	`[{"id":"p1","qty":3}]`
#2	Visible Test Case	`[[{"id":"p1","qty":2}, {"id":"p2","qty":1}], {"type":"REMOVE","id":"p1"}]`	`[{"id":"p2","qty":1}]`
#3	Hidden Test Case	`[[{"id":"p1","qty":2}], {"type":"ADD","id":"p1","qty":-2}]`	`[]`
#4	Hidden Test Case	`[[], {"type":"ADD","id":"p9","qty":1}]`	`[{"id":"p9","qty":1}]`

ชุดที่ 5 • ข้อที่ 2/3 Memoized Job Filter & Sort Selector

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(jobs, filter)` สำหรับคำนวณรายการแสดงผลใน React UI โดยกรองเฉพาะงานที่:
1. `job.salary >= filter.minSalary` (ถ้ามีระบุ)
2. `job.title` มีคำว่า `filter.search` (ไม่สนใจตัวพิมพ์เล็ก-ใหญ่)
และคืนค่าเป็นอาร์เรย์ของ `job.id` เรียงตาม `salary` จากมากไปน้อย (หากเท่ากันให้เรียงตาม `id` A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(jobs, filter) {
  const f = filter || {};
  const minSal = Number(f.minSalary) || 0;
  const q = String(f.search || '').trim().toLowerCase();
  return (jobs || [])
    .filter((j) => {
      if (!j) return false;
      if ((Number(j.salary) || 0) < minSal) return false;
      if (q && !String(j.title || '').toLowerCase().includes(q)) return false;
      return true;
    })
    .sort((a, b) => {
      if (b.salary !== a.salary) return b.salary - a.salary;
      return String(a.id).localeCompare(String(b.id));
    })
    .map((j) => j.id);
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `กรองข้อมูลตามเงื่อนไขค้นหาและเงินเดือนขั้นต่ำ แล้วจัดเรียงลำดับบนสำเนาอาร์เรย์เพื่อคืนค่าเฉพาะรายการ ID`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"id":"j1","title":"React Developer","salary":50000},{"id":"j2","title":"Senior React Engineer","salary":80000},{"id":"j3","title":"Python Engineer","salary":90000}],{"search":"react","minSalary":45000}]`	`["j2","j1"]`
#2	Visible Test Case	`[[{"id":"b","title":"Frontend","salary":60000},{"id":"a","title":"Frontend Lead","salary":60000}],{"search":"frontend","minSalary":60000}]`	`["a","b"]`
#3	Hidden Test Case	`[[{"id":"j1","title":"React","salary":30000}],{"search":"react","minSalary":50000}]`	`[]`

ชุดที่ 5 • ข้อที่ 3/3 **Hierarchical Comment Tree Builder** HARD 35 คะแนน JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `solution(comments)` ที่แปลงรายการคอมเมนต์แบบแบน `[{ id, parentId, text }]` ให้เป็นโครงสร้างต้นไม้ `[{ id, text, children: [...] }]` สำหรับเรนเดอร์ Recursive Component ใน React โดยคอมเมนต์ระดับบนสุดจะมี `parentId: null`

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(comments) {
  const map = new Map();
  const roots = [];
  for (const c of comments || []) {
    map.set(c.id, { id: c.id, text: c.text, children: [] });
  }
  for (const c of comments || []) {
    const node = map.get(c.id);
    if (c.parentId && map.has(c.parentId)) {
      map.get(c.parentId).children.push(node);
    } else {
      roots.push(node);
    }
  }
  return roots;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `สร้าง Map ของโหนดทั้งหมดในรอบแรก (`O(n)`) แล้วเชื่อมโหนดลูกเข้ากับ `children` ของ `parentId` ในรอบที่สอง`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — React** และอัปเดตคะแนนแกน **FRONTEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[{"id":"c1","parentId":null,"text":"Root"},{"id":"c2","parentId":"c1","text":"Reply"}]`	`[{"id":"c1","text":"Root","children":[{"id":"c2","text":"Reply","children":[]}]}]`
#2	Visible Test Case	`[{"id":"1","parentId":null,"text":"A"},{"id":"2","parentId":null,"text":"B"}]`	`[{"id":"1","text":"A","children":[]},{"id":"2","text":"B","children":[]}]`
#3	Hidden Test Case	`[[]]`	`[]`

ชุดข้อสอบที่ 6 | MDN Web Docs | สอบทฤษฎี (ความรู้ประกอบ) | เวลา 25 นาที • เกณฑ์ผ่าน 75% • ทักษะ: **Application Security (SECURITY)**

Web Security, HTTP & Browser Standards (MDN Web Docs Reference)

อ้างอิงเอกสารมาตรฐาน MDN Web Docs (developer.mozilla.org) ด้าน Web Security, HTTP Protocol, CORS และ Authentication (ผลสอบทฤษฎีใช้เพื่อการเรียนรู้ทบทวน)

ชุดที่ 6 • ข้อที่ 1/5 HttpOnly Cookie Protection

EASY | 20 คะแนน | ปนัย (Multiple Choice)

คำถามปนัย (Question Prompt):

ตามมาตรฐาน MDN Web Security การกำหนดแฟล็ก `HttpOnly` ให้กับคุกกี้ที่เก็บ Session หรือ Refresh Token ช่วยป้องกันการโจมตีรูปแบบใดโดยตรง?

ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: ป้องกัน JavaScript ฝั่งไคลเอนต์เข้าถึงค่าคุกกี้ผ่าน document.cookie เมื่อเกิดช่องโหว่ XSS** ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: ป้องกันการแก้ไขคำสั่ง SQL ในฐานข้อมูลผ่านพารามิเตอร์ของ URL
- ○ ตัวเลือกที่ 3: บังคับให้เบราว์เซอร์บีบอัดข้อมูลคุกกี้ด้วยอัลกอริทึม Gzip ทุกครั้ง
- ○ ตัวเลือกที่ 4: แปลงโปรโตคอล HTTP เป็น WebSocket โดยอัตโนมัติ

คำอธิบายเฉลย: HttpOnly ป้องกันไม่ให้สคริปต์ฝั่งไคลเอนต์เข้าถึงคุกกี้ผ่าน document.cookie จึงลดความเสียหายจาก XSS

ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- เมื่อคลิกเลือกตัวเลือก **“ป้องกัน JavaScript ฝั่งไคลเอนต์เข้าถึงค่าคุกกี้ผ่าน document.cookie เมื่อเกิดช่องโหว่ XSS”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **20 คะแนนเต็ม**

ชุดที่ 6 • ข้อที่ 2/5 CORS Preflight Request Method

EASY | 20 คะแนน | ปนัย (Multiple Choice)

คำถามปนัย (Question Prompt):

ตามเอกสาร MDN เรื่อง Cross-Origin Resource Sharing (CORS) เมื่อเว็บแอปพลิเคชันส่งคำขอแบบ `PUT` หรือแนบ Header `Authorization` ข้ามโดเมน เบราว์เซอร์จะส่ง Preflight Request ด้วย HTTP Method ใดไปตรวจสอบสิทธิ์ก่อน?

ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: HTTP OPTIONS (Preflight Request)** ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: HTTP HEAD
- ○ ตัวเลือกที่ 3: HTTP TRACE
- ○ ตัวเลือกที่ 4: HTTP CONNECT

คำอธิบายเฉลย: เบราว์เซอร์ส่ง HTTP OPTIONS เพื่อตรวจสอบ Access-Control-Allow-Methods และ Access-Control-Allow-Headers

ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- เมื่อคลิกเลือกตัวเลือก **“HTTP OPTIONS (Preflight Request)”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **20 คะแนนเต็ม**

ชุดที่ 6 • ข้อที่ 3/5 Content-Security-Policy (CSP) Defense

MEDIUM | 20 คะแนน | ปรนัย (Multiple Choice)

📋 **คำถามปรนัย (Question Prompt):**

HTTP Response Header ใดตามมาตรฐาน MDN ที่ใช้จำกัดแหล่งที่มาของสคริปต์ รูปภาพ และสไตล์ชีตที่เบราว์เซอร์ได้รับอนุญาตให้โหลดและรันบนหน้าเว็บ?

🛠️ **ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)**

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: Content-Security-Policy** — ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: X-Powered-By
- ○ ตัวเลือกที่ 3: Accept-Encoding
- ○ ตัวเลือกที่ 4: Transfer-Encoding

💡 **คำอธิบายเฉลย:** Content-Security-Policy (CSP) ใช้กำหนดโดเมนที่ได้รับอนุญาต เช่น script-src 'self'

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- เมื่อคลิกเลือกตัวเลือก **“Content-Security-Policy”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **20 คะแนนเต็ม**

ชุดที่ 6 • ข้อที่ 4/5 HTTP 401 vs 403 Semantics

MEDIUM | 20 คะแนน | ปรนัย (Multiple Choice)

📋 **คำถามปรนัย (Question Prompt):**

ตามนิยาม HTTP Status Codes บน MDN ความแตกต่างที่ถูกต้องระหว่าง `401 Unauthorized` และ `403 Forbidden` คือข้อใด?

🛠️ **ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)**

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: 401 คือยังไม่ได้ยืนยันตัวตนหรือ Token ไม่ถูกต้อง ส่วน 403 คือยืนยันตัวตนแล้วแต่บทบาทไม่มีสิทธิ์เข้าถึง** — ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: 401 ใช้กับข้อผิดพลาดฝั่งฐานข้อมูล ส่วน 403 ใช้เมื่อเซิร์ฟเวอร์ปลายทางล่ม
- ○ ตัวเลือกที่ 3: 401 ใช้เฉพาะกับคำขอแบบ GET ส่วน 403 ใช้เฉพาะกับคำขอแบบ POST
- ○ ตัวเลือกที่ 4: 401 หมายถึงไม่พบหน้าเว็บ ส่วน 403 หมายถึงส่งคำขอถี่เกินกำหนด

💡 **คำอธิบายเฉลย:** 401 หมายถึงยังไม่ผ่านการยืนยันตัวตน (Unauthenticated) ส่วน 403 คือยืนยันตัวตนแล้วแต่ไม่มีสิทธิ์เข้าถึงทรัพยากรนั้น

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- เมื่อคลิกเลือกตัวเลือก **“401 คือยังไม่ได้ยืนยันตัวตนหรือ Token ไม่ถูกต้อง ส่วน 403 คือยืนยันตัวตนแล้วแต่บทบาทไม่มีสิทธิ์เข้าถึง”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **20 คะแนนเต็ม**

ชุดที่ 6 • ข้อที่ 5/5 **SameSite Cookie Attribute for CSRF Mitigation** MEDIUM 20 คะแนน ปรนัย (Multiple Choice)

📋 **คำถามปรนัย (Question Prompt):**

การกำหนดค่าแอตทริบิวต์ `SameSite=Lax` หรือ `SameSite=Strict` บนคุกกี้ตามแนวทางของ MDN มีวัตถุประสงค์หลักเพื่อลดความเสี่ยงจากการโจมตีประเภทใด?

🛠️ **ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)**

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- 🔘 **ตัวเลือกที่ 1: Cross-Site Request Forgery (CSRF)** ✓ คำตอบที่ถูกต้อง
- ตัวเลือกที่ 2: Distributed Denial of Service (DDoS)
- ตัวเลือกที่ 3: Buffer Overflow ในหน่วยความจำของเซิร์ฟเวอร์
- ตัวเลือกที่ 4: DNS Cache Poisoning

💡 **คำอธิบายเฉลย:** SameSite ควบคุมการส่งคุกกี้ข้ามเว็บไซต์ จึงช่วยป้องกัน Cross-Site Request Forgery (CSRF)

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- เมื่อคลิกเลือกตัวเลือก **“Cross-Site Request Forgery (CSRF)”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **20 คะแนนเต็ม**
- **ผลลัพธ์รวมของชุดข้อสอบทฤษฎี**: แสดงหน้าสรุปผล **คะแนนสุทธิ (Final Score): 100% (ผ่านเกณฑ์มาตรฐาน ≥75%)** บันทึกลงในตารางประวัติการเข้าสอบ โดยไม่ไปแก้ไขคะแนนทักษะปฏิบัติบนกราฟเรดาร์

หมวด C: คลังข้อสอบอ้างอิงจาก W3Schools.com (3 ชุดข้อสอบ)

อ้างอิงบทเรียน SQL/PostgreSQL, Python Data Analytics และการจัดการ Docker & DevOps Configuration

ชุดข้อสอบที่ 7 | W3Schools.com | สอบปฏิบัติ (อัปเดต Global Skill Score) | เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **PostgreSQL (DATABASE)**

PostgreSQL & Relational Query Logic (W3Schools Reference)

อ้างอิงบทเรียน SQL & PostgreSQL จาก W3Schools.com วัดทักษะการเชื่อมตาราง (INNER JOIN), การจัดกลุ่มพร้อมเงื่อนไข (GROUP BY / HAVING) และการจัดอันดับข้อมูล (Window Ranking)

ชุดที่ 7 • ข้อที่ 1/3 Relational INNER JOIN Matcher

EASY | 30 คะแนน | JavaScript (Node.js)

โจทย์คำถาม (Problem Prompt):

อ้างอิงบทเรียน SQL INNER JOIN จาก W3Schools: เขียนฟังก์ชัน `solution(users, orders)` ที่จำลองคำสั่ง:
`SELECT orders.orderId, users.name, orders.total FROM orders INNER JOIN users ON orders.userId = users.id ORDER BY orders.orderId ASC`
คืนค่าอาร์เรย์ของ `{ orderId, name, total }` เฉพาะรายการที่หา `userId` เจอในตาราง `users`

ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(users, orders) {
  const userMap = new Map((users || []).map((u) => [u.id, u.name]));
  return (orders || [])
    .filter((o) => o && userMap.has(o.userId))
    .sort((a, b) => a.orderId - b.orderId)
    .map((o) => ({ orderId: o.orderId, name: userMap.get(o.userId), total: o.total }));
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `สร้าง Hash Map ของ `users` เพื่อทำการ Hash Join กับ `orders` ในเวลา `O(n + m)` แล้วเรียงตาม `orderId``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"}],[{"orderId":102,"userId":1,"total":500},{"orderId":101,"userId":2,"total":300},{"orderId":103,"userId":99,"total":100}]]`	`[{"orderId":101,"name":"Bob","total":300},{"orderId":102,"name":"Alice","total":500}]`
#2	Visible Test Case	`[[{"id":5,"name":"Chai"}],[{"orderId":1,"userId":5,"total":1200}]]`	`[{"orderId":1,"name":"Chai","total":1200}]`
#3	Hidden Test Case	`[[{"id":1,"name":"Alice"}],[{"orderId":10,"userId":2,"total":50}]]`	`[]`

ชุดที่ 7 • ข้อที่ 2/3 GROUP BY Category with HAVING Clause

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

อ้างอิงบทเรียน SQL HAVING Clause จาก W3Schools: เขียนฟังก์ชัน `solution(sales, minRevenue)` ที่จำลองคำสั่ง: `SELECT category, SUM(amount) AS totalRevenue, COUNT(*) AS orderCount FROM sales WHERE status = 'PAID' GROUP BY category HAVING SUM(amount) >= minRevenue ORDER BY totalRevenue DESC`

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(sales, minRevenue) {
  const groups = new Map();
  for (const s of sales || []) {
    if (!s || s.status !== 'PAID') continue;
    const cur = groups.get(s.category) || { category: s.category, totalRevenue: 0, orderCount: 0 };
    cur.totalRevenue += Number(s.amount || 0);
    cur.orderCount += 1;
    groups.set(s.category, cur);
  }
  return Array.from(groups.values())
    .filter((g) => g.totalRevenue >= Number(minRevenue || 0))
    .sort((a, b) => b.totalRevenue - a.totalRevenue || a.category.localeCompare(b.category));
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `กรองแถวด้วยเงื่อนไข `WHERE status = 'PAID'` ก่อนจัดกลุ่ม แล้วจึงกรองผลรวมด้วยเงื่อนไข `HAVING``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"category":"Cloud","amount":600,"status":"PAID"}, {"category":"Cloud","amount":500,"status":"PAID"}, {"category":"Book","amount":200,"status":"PAID"}, {"category":"Book","amount":900,"status":"CANCELLED"}], 1000]`	`[{"category":"Cloud","totalRevenue":1100,"orderCount":2}]`
#2	Visible Test Case	`[[{"category":"A","amount":500,"status":"PAID"}, {"category":"B","amount":800,"status":"PAID"}], 400]`	`[{"category":"B","totalRevenue":800,"orderCount":1}, {"category":"A","totalRevenue":500,"orderCount":1}]`
#3	Hidden Test Case	`[[], 100]`	`[]`

ชุดที่ 7 • ข้อที่ 3/3 Window Function Top-N Per Department

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

อ้างอิงการทำ Partition Ranking ใน PostgreSQL: เขียนฟังก์ชัน `solution(employees, topN)` ที่จำลอง `ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC, name ASC)` เพื่อดึงพนักงานที่มีเงินเดือนสูงสุดไม่เกิน `topN` อันดับแรกของแต่ละแผนก โดยเรียงผลลัพธ์ตามชื่อแผนก (A-Z) และเงินเดือนจากมากไปน้อย

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(employees, topN) {
  const limit = Math.max(0, Number(topN) || 0);
  const byDept = new Map();
  for (const emp of employees || []) {
    if (!emp || !emp.department) continue;
    const list = byDept.get(emp.department) || [];
    list.push({ department: emp.department, name: emp.name, salary: Number(emp.salary || 0) });
    byDept.set(emp.department, list);
  }
  const out = [];
  for (const dept of Array.from(byDept.keys()).sort()) {
    const sorted = byDept.get(dept).sort((a, b) => b.salary - a.salary || a.name.localeCompare(b.name));
    out.push(...sorted.slice(0, limit));
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `แบ่งกลุ่มข้อมูลตาม `department` (Partition) เรียงลำดับตาม `salary DESC, name ASC` แล้วตัดมาเฉพาะ `topN` แถวแรกของแต่ละกลุ่ม`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — PostgreSQL** และอัปเดตคะแนนแกน **DATABASE** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[[{"department":"Eng","name":"Ann","salary":70000}, {"department":"Eng","name":"Ben","salary":90000}, {"department":"Eng","name":"Cid","salary":80000}, {"department":"HR","name":"May","salary":55000}], 2]`	`[{"department":"Eng","name":"Ben","salary":90000}, {"department":"Eng","name":"Cid","salary":80000}, {"department":"HR","name":"May","salary":55000}]`
#2	👁 Visible Test Case	`[[{"department":"Data","name":"Zoe","salary":85000}, {"department":"Data","name":"Amy","salary":85000}], 1]`	`[{"department":"Data","name":"Amy","salary":85000}]`
#3	🔒 Hidden Test Case	`[[], 3]`	`[]`

ชุดข้อสอบที่ 8 W3Schools.com สอบปฏิบัติ (อัปเดต Global Skill Score) ⏱ เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **Python (BACKEND)**

Python Data Processing & Analytics (W3Schools Reference)

อ้างอิงบทเรียน Python Tutorial & Data Structures จาก W3Schools.com วัดทักษะการทำความสะอาดข้อมูล (Data Cleaning), การแปลงเมทริกซ์ และการวิเคราะห์ Log เชิงสถิติด้วยภาษา Python

ชุดที่ 8 • ข้อที่ 1/3 Clean & Summarize Sensor Readings

EASY | 30 คะแนน | Python 3

📋 **โจทย์คำถาม (Problem Prompt):**

อ้างอิงบทเรียน Python Lists & Math จาก W3Schools: เขียนฟังก์ชัน `def solution(readings):` ในภาษา Python ที่รับลิสต์ของค่าเซนเซอร์ (อาจมีค่า `None` หรือค่าติดลบซึ่งเป็นข้อมูลผิดพลาด) ให้กรองเฉพาะค่าตัวเลขที่มีค่า `>= 0` แล้วคืนค่า dictionary `{"count": จำนวนข้อมูลที่ถูกต้อง, "average": ค่าเฉลี่ยปัดเศษทศนิยม 2 ตำแหน่ง}` (หากไม่มีข้อมูลที่ถูกต้องเลยให้ `average` เป็น `0`)

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
def solution(readings):
    valid = [float(x) for x in (readings or []) if isinstance(x, (int, float)) and not isinstance(x, bool)
and x >= 0]
    if not valid:
        return {"count": 0, "average": 0}
    avg = round(sum(valid) / len(valid), 2)
    if avg.is_integer():
        avg = int(avg)
    return {"count": len(valid), "average": avg}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้ List Comprehension กรองค่า `None` และค่าติดลบออก แล้วคำนวณค่าเฉลี่ยด้วย `sum(valid) / len(valid)`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[10, 20, null, -5, 30]]`	`{"count":3,"average":20}`
#2	Visible Test Case	`[[1.5, 2.25, 3.0]]`	`{"count":3,"average":2.25}`
#3	Hidden Test Case	`[[null, -10, -1]]`	`{"count":0,"average":0}`
#4	Hidden Test Case	`[[]]`	`{"count":0,"average":0}`

ชุดที่ 8 • ข้อที่ 2/3 **2D Matrix Transpose** MEDIUM 35 คะแนน Python 3

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `def solution(matrix):` ในภาษา Python ที่รับเมทริกซ์ 2 มิติขนาด `R x C` แล้วคืนค่าเมทริกซ์ที่สลับแถวเป็นคอลัมน์ (Transpose ขนาด `C x R`) หากเมทริกซ์ว่างให้คืนค่า `[]`

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
def solution(matrix):
    if not matrix or not matrix[0]:
        return []
    return [list(col) for col in zip(*matrix)]
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้ `zip(*matrix)` ร่วมกับ List Comprehension เพื่อสลับแถวและคอลัมน์ใน Python อย่างกระชับ`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[[1, 2, 3], [4, 5, 6]]]`	`[[1,4],[2,5],[3,6]]`
#2	Visible Test Case	`[[[10, 20], [30, 40]]]`	`[[10,30],[20,40]]`
#3	🔒 Hidden Test Case	`[[[7]]]`	`[[7]]`
#4	🔒 Hidden Test Case	`[[]]`	`[]`

ชุดที่ 8 • ข้อที่ 3/3 Service Error Rate Spike Detector

HARD | 35 คะแนน | Python 3

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `def solution(logs, thresholdPercent):` ในภาษา Python ที่รับลิสต์ของ Log `{"service": str, "status": int}` และคำนวณสัดส่วน Error (`status >= 500`) ต่อจำนวน Log ทั้งหมดของแต่ละบริการ คืนค่ารายชื่อ `service` ที่มีอัตรา Error `>= thresholdPercent` เรียงตามตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
def solution(logs, thresholdPercent):
    totals = {}
    errors = {}
    for entry in (logs or []):
        if not isinstance(entry, dict) or 'service' not in entry:
            continue
        svc = str(entry['service'])
        status = int(entry.get('status', 200))
        totals[svc] = totals.get(svc, 0) + 1
        if status >= 500:
            errors[svc] = errors.get(svc, 0) + 1
    flagged = []
    for svc, total in totals.items():
        err_rate = (errors.get(svc, 0) / total) * 100.0
        if err_rate >= float(thresholdPercent):
            flagged.append(svc)
    return sorted(flagged)
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `นับจำนวน Request ทั้งหมดและจำนวนที่ `status >= 500` แยกตามบริการด้วย Dictionary แล้วเปรียบเทียบเปอร์เซ็นต์กับ `thresholdPercent``
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — Python** และอัปเดตคะแนนแกน **BACKEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"service":"auth","status":500}, {"service":"auth","status":200}, {"service":"pay","status":503}, {"service":"pay","status":500}, {"service":"search","status":200}], 50]`	`["auth","pay"]`
#2	Visible Test Case	`[[{"service":"api","status":200}, {"service":"api","status":500}], 80]`	`[]`
#3	Hidden Test Case	`[[], 10]`	`[]`

ชุดข้อสอบที่ 9 W3Schools.com สอบปฏิบัติ (อัปเดต Global Skill Score) ⏱ เวลา 45 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **Docker (DEVOPS)**

Docker & DevOps Configuration Linter (W3Schools Reference)

อ้างอิงแนวทาง Container & Cloud Configuration จาก W3Schools และมาตรฐาน DevOps วัดทักษะการแปลงไฟล์ .env, การตรวจสอบความปลอดภัยของ Dockerfile และการจัดลำดับ Container Dependencies

ชุดที่ 9 • ข้อที่ 1/3 Docker Environment File (.env) Parser

EASY 30 คะแนน JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `solution(envText)` ที่แปลงข้อความในไฟล์ `.env` สำหรับ Docker Container ให้เป็นอ็อบเจกต์ `{ KEY: VALUE }` โดยมีเกณฑ์:
- ข้ามบรรทัดว่างและบรรทัดคอมเมนต์ที่ขึ้นต้นด้วย `#`
- ตัดช่องว่างรอบคีย์และค่า รวมถึงตัดเครื่องหมาย `"` หรือ `'` ที่ครอบหัวท้ายของค่าออก

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(envText) {
  const out = {};
  const lines = String(envText || '').split(/\r?\n/);
  for (const raw of lines) {
    const line = raw.trim();
    if (!line || line.startsWith('#')) continue;
    const eqIdx = line.indexOf('=');
    if (eqIdx <= 0) continue;
    const key = line.slice(0, eqIdx).trim();
    let val = line.slice(eqIdx + 1).trim();
    if ((val.startsWith('"') && val.endsWith('"')) || (val.startsWith("'") && val.endsWith("'"))) {
      val = val.slice(1, -1);
    }
    out[key] = val;
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `แยกบรรทัดด้วย `\r?\n` ข้ามบรรทัด `#` และใช้ `indexOf('=')` เพื่อให้รองรับค่าที่มีเครื่องหมาย `=` อยู่ภายใน`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **30 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`["# Database Config\nPORT=8080\nNODE_ENV=\"production\"\n"]`	`{"PORT":"8080","NODE_ENV":"production"}`
#2	Visible Test Case	`["APP_NAME='smart-career'\n# comment\nLOG_LEVEL=info"]`	`{"APP_NAME":"smart-career","LOG_LEVEL":"info"}`
#3	Hidden Test Case	`["   \n# only comments"]`	`{}`

ชุดที่ 9 • ข้อที่ 2/3 Dockerfile Security & Best-Practice Linter

MEDIUM | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(lines)` ที่รับอาร์เรย์ของบรรทัดคำสั่งใน `Dockerfile` และตรวจหาความเสี่ยงตามกฎต่อไปนี้:
- หากมีคำสั่ง `FROM` ที่ใช้แท็ก `:latest` หรือไม่ระบุแท็กเวอร์ชันเลย ให้เพิ่มรหัส `"AVOID_LATEST_TAG"`
- หากสถานะผู้ใช้สุดท้ายคือ `USER root` ให้เพิ่มรหัส `"AVOID_ROOT_USER"`
- หากมีคำสั่งขึ้นต้นด้วย `ADD` ให้เพิ่มรหัส `"PREFER_COPY_OVER_ADD"`
คืนค่ารหัสความเสี่ยงที่ไม่ซ้ำกันเรียงตามตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(lines) {
  const issues = new Set();
  let lastUser = null;
  for (const raw of lines || []) {
    const line = String(raw || '').trim();
    if (!line || line.startsWith('#')) continue;
    if (/^FROM\s+/i.test(line)) {
      const image = line.replace(/^FROM\s+/i, '').split(/\s+/)[0];
      if (!image.includes(':') || image.endsWith(':latest')) {
        issues.add('AVOID_LATEST_TAG');
      }
    } else if (/^USER\s+/i.test(line)) {
      lastUser = line.replace(/^USER\s+/i, '').trim().toLowerCase();
    } else if (/^ADD\s+/i.test(line)) {
      issues.add('PREFER_COPY_OVER_ADD');
    }
  }
  if (lastUser === 'root') {
    issues.add('AVOID_ROOT_USER');
  }
  return Array.from(issues).sort();
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ตรวจสอบแต่ละคำสั่งใน Dockerfile เพื่อบังคับใช้การ Pin เวอร์ชันอิมเมจ การใช้ COPY แทน ADD และการไม่รัน Container ด้วยสิทธิ์ root`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **35 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[["FROM node:latest", "ADD . /app", "USER root"]]`	`["AVOID_LATEST_TAG","AVOID_ROOT_USER","PREFER_COPY_OVER_ADD"]`
#2	Visible Test Case	`[["FROM node:20-alpine", "COPY . /app", "USER node"]]`	`[]`
#3	Hidden Test Case	`[["FROM ubuntu", "USER root", "USER appuser"]]`	`["AVOID_LATEST_TAG"]`

ชุดที่ 9 • ข้อที่ 3/3 Docker Compose Startup Order Resolver

HARD | 35 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(services)` ที่รับอ็อบเจกต์ของบริการใน Docker Compose เช่น `{ web: ["api"], api: ["db", "redis"], db: [], redis: [] }` (แต่ละคีย์ระบุรายการบริการที่ต้องสตาร์ทก่อนใน `depends_on`) ให้คืนค่าลำดับการสตาร์ทบริการที่ถูกต้อง (หากมีหลายบริการที่พร้อมสตาร์ทพร้อมกัน ให้เรียงตามชื่อบริการ A-Z) หากพบวงจรพึ่งพากันเอง (Cycle) ให้คืนค่า `["CYCLE_DETECTED"]`

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 3”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(services) {
  const graph = services && typeof services === 'object' ? services : {};
  const names = Object.keys(graph).sort();
  const resolved = new Set();
  const order = [];
  let progress = true;
  while (order.length < names.length && progress) {
    progress = false;
    for (const name of names) {
      if (resolved.has(name)) continue;
      const deps = Array.isArray(graph[name]) ? graph[name] : [];
      if (deps.every((d) => resolved.has(d))) {
        resolved.add(name);
        order.push(name);
        progress = true;
        break;
      }
    }
  }
  if (order.length < names.length) return ['CYCLE_DETECTED'];
  return order;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้อัลกอริทึม Topological Sort เลือกบริการที่ Dependencies ทั้งหมดพร้อมแล้วตามลำดับตัวอักษร หากไม่สามารถเลือกต่อได้จนครบแสดงว่ามี Circular Dependency`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 3 ✓”** และได้คะแนนเต็ม **35 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — Docker** และอัปเดตคะแนนแกน **DEVOPS** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[{"web":["api"],"api":["db","redis"],"db":[],"redis":[]}]`	`["db","redis","api","web"]`
#2	Visible Test Case	`[{"a":["b"],"b":["a"]}]`	`["CYCLE_DETECTED"]`
#3	Hidden Test Case	`[{"worker":["queue"],"queue":[]}]`	`["queue","worker"]`

หมวด D: คลังข้อสอบมาตรฐานพื้นฐานเดิมในระบบ (4 ชุดข้อสอบ)

ชุดข้อสอบพื้นฐานสำหรับตรวจสอบความเข้าใจเบื้องต้นด้าน JavaScript, SQL, Python และทฤษฎีวิศวกรรมเว็บ

ชุดข้อสอบที่ 10 | SmartCareer Core | สอบปฏิบัติ (อัปเดต Global Skill Score)

⏱ เวลา 30 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **JavaScript (FRONTEND)**

JavaScript Practical Coding Assessment

วัดทักษะการเขียนโค้ด JavaScript เชิงปฏิบัติสำหรับการประมวลผลข้อมูลและตรรกะพื้นฐาน

ชุดที่ 10 • ข้อที่ 1/2 Sum Positive Numbers

EASY | 50 คะแนน | JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `solution(numbers)` ที่รับอาร์เรย์ของตัวเลข และคืนค่าผลรวมของตัวเลขที่มีค่ามากกว่า 0 เท่านั้น หากไม่มีตัวเลขบวกเลยให้คืนค่า 0

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(numbers) {
  if (!Array.isArray(numbers)) return 0;
  return numbers.filter((n) => typeof n === 'number' && n > 0).reduce((a, b) => a + b, 0);
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `กรองเฉพาะค่าที่มากกว่า 0 ด้วย filter แล้วหาผลรวมด้วย reduce`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **50 คะแนน**

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[1, -2, 3, 4]]`	`8`
#2	Visible Test Case	`[[-5, 0, -1]]`	`0`
#3	🔒 Hidden Test Case	`[[10, 20, -5, 15]]`	`45`
#4	🔒 Hidden Test Case	`[[]]`	`0`

ชุดที่ 10 • ข้อที่ 2/2 Normalize Unique Tags

MEDIUM | 50 คะแนน | JavaScript (Node.js)

📋 โจทย์คำถาม (Problem Prompt):

เขียนฟังก์ชัน `solution(tags)` ที่รับอาร์เรย์ของสตริง แปลงทุกตัวเป็นตัวพิมพ์เล็ก ตัดช่องว่างหัวท้าย ตัดสตริงว่างทิ้ง และคืนค่าอาร์เรย์ของแท็กที่ไม่ซ้ำกันเรียงตามลำดับตัวอักษร (A-Z)

🛠️ ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)

1. คลิกปุ่มแท็บ **“ข้อ 2”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(tags) {
  if (!Array.isArray(tags)) return [];
  const set = new Set(tags.map((t) => String(t).trim().toLowerCase()).filter(Boolean));
  return Array.from(set).sort();
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `ใช้ trim(), toLowerCase(), Set เพื่อตัดซ้ำ และ sort() เพื่อเรียงลำดับ`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 2 ✓”** และได้คะแนนเต็ม **50 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — JavaScript** และอัปเดตคะแนนแกน **FRONTEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	👁 Visible Test Case	`[[" React ", "nodejs", "react", "  "]]`	`["nodejs","react"]`
#2	👁 Visible Test Case	`[["SQL", "api", "Api"]]`	`["api","sql"]`
#3	🔒 Hidden Test Case	`[["docker", "Docker", "kubernetes", "aws"]]`	`["aws","docker","kubernetes"]`
#4	🔒 Hidden Test Case	`[[]]`	`[]`

ชุดข้อสอบที่ 11 SmartCareer Core สอบปฏิบัติ (อัปเดต Global Skill Score) ⏱ เวลา 30 นาที • เกณฑ์ผ่าน 60% • ทักษะ: **SQL (DATABASE)**

SQL & Data Query Logic Assessment

ประเมินทักษะการกรองและสรุปผลข้อมูลเชิงตารางสำหรับงานฐานข้อมูลและ Backend

ชุดที่ 11 • ข้อที่ 1/1 **Aggregate Department Salary** MEDIUM 100 คะแนน JavaScript (Node.js)

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `solution(rows, minSalary)` ที่จำลองคำสั่ง `SELECT department, COUNT(*) FROM employees WHERE salary >= minSalary GROUP BY department` โดยรับ `rows` เป็นอาร์เรย์ของ `{ department, salary }` และคืนค่าอ็อบเจกต์ที่เก็บจำนวนพนักงานต่อแผนกที่มีเงินเดือนตั้งแต่ `minSalary` ขึ้นไป

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
function solution(rows, minSalary) {
  const out = {};
  for (const r of rows || []) {
    if (r && typeof r.salary === 'number' && r.salary >= minSalary) {
      out[r.department] = (out[r.department] || 0) + 1;
    }
  }
  return out;
}
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `กรองแถวที่ salary >= minSalary แล้วนับจำนวนแยกตาม department`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **4 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 2 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **100 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥60%)** ได้รับเหรียญ **SmartCareer Verified — SQL** และอัปเดตคะแนนแกน **DATABASE** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[[{"department":"Eng","salary":50000},{"department":"Eng","salary":30000},{"department":"HR","salary":45000}],40000]`	`{"Eng":1,"HR":1}`
#2	Visible Test Case	`[[{"department":"Data","salary":60000},{"department":"Data","salary":75000}],50000]`	`{"Data":2}`
#3	🔒 Hidden Test Case	`[[{"department":"Sales","salary":25000}],40000]`	`{}`
#4	🔒 Hidden Test Case	`[[], 30000]`	`{}`

ชุดข้อสอบที่ 12 | SmartCareer Core | สอบปฏิบัติ (อัปเดต Global Skill Score)

⏱ เวลา 30 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **Python (BACKEND)**

Python Practical Problem Solving

วัดทักษะการเขียนโปรแกรมภาษา Python สำหรับการประมวลผลข้อมูลและอัลกอริทึม

ชุดที่ 12 • ข้อที่ 1/1 Word Frequency Counter

EASY | 100 คะแนน | Python 3

📋 **โจทย์คำถาม (Problem Prompt):**

เขียนฟังก์ชัน `def solution(words):` ในภาษา Python ที่รับลิสต์ของคำศัพท์และคืนค่า dictionary ที่นับความถี่ของแต่ละคำ (แปลงเป็นตัวพิมพ์เล็กทั้งหมด)

🛠️ **ขั้นตอนการกดทดสอบแบบจับมือทำ (Step-by-Step Actions)**

1. คลิกปุ่มแท็บ **“ข้อ 1”** ทางด้านซ้ายของห้องสอบ (สังเกตโค้ดตั้งต้น `Starter Code` ในกล่องดำ)
2. **ทดสอบกรณีโค้ดยังไม่สมบูรณ์ (Negative Test)**: กดปุ่ม **“▶ รันทดสอบ Visible Test Cases”** ทันทีโดยยังไม่แก้โค้ด เพื่อตรวจว่าระบบดักจับคำตอบที่ผิดได้
3. **ทดสอบกรณีโค้ดถูกต้อง (Positive Test)**: ลบโค้ดเดิมในกล่อง Code Editor ออกทั้งหมด แล้วคัดลอก **โค้ดเฉลย (Solution Code)** ด้านล่างนี้ไปวางแทน:

```
def solution(words):
    counts = {}
    for w in (words or []):
        k = str(w).strip().lower()
        if k:
            counts[k] = counts.get(k, 0) + 1
    return counts
```

4. พิมพ์ข้อความในช่อง *“คำอธิบายแนวคิดการเขียนโค้ด (Candidate Explanation)”* เช่น `วนลูปแปลงคำเป็นตัวพิมพ์เล็กและนับจำนวนใน dictionary`
5. คลิกปุ่ม **“▶ รันทดสอบ Visible Test Cases”** เพื่อตรวจผลลัพธ์เบื้องต้น จากนั้นคลิกปุ่มสีน้ำเงิน **“บันทึกและส่งคำตอบข้อนี้”**
6. เมื่อทำครบทุกข้อแล้ว ให้คลิกปุ่มสีน้ำเงินด้านบนสุด **“ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”** เพื่อสรุปคะแนนทั้งชุด

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- **จังหวะกดรันด้วย Starter Code เดิม (Negative Test)**: กล่องผลลัพธ์ด้านล่างแสดงแถบสีแดง `FAILED` เพราะโค้ดตั้งต้นยังไม่ได้คำนวณผลลัพธ์จริง
- **จังหวะกดรันหลังวางโค้ดเฉลย (Visible Test Cases)**: แสดงข้อความ **“ผลรันชุดทดสอบตัวอย่าง: ผ่าน 2/2 ข้อ”** พร้อมแถบสีเขียว `PASSED` ครบทุกข้อตัวอย่าง
- **จังหวะกด “บันทึกและส่งคำตอบข้อนี้”**: ระบบรันตรวจรวม **3 ชุดทดสอบ** (Visible 2 ข้อ + Hidden 1 ข้อ) ปุ่มแท็บด้านซ้ายเปลี่ยนเป็นสีเขียว **“ข้อ 1 ✓”** และได้คะแนนเต็ม **100 คะแนน**
- **จังหวะกด “ยืนยันส่งข้อสอบปฏิบัติ (Finalize)”**: หน้าจอเปลี่ยนเป็นหน้าสรุปผล แสดงป้ายสีเขียว **“ส่งข้อสอบเรียบร้อยแล้ว”** คะแนนสุทธิ **100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** ได้รับเหรียญ **SmartCareer Verified — Python** และอัปเดตคะแนนแกน **BACKEND** บนกราฟเรดาร์ในหน้าโปรไฟล์ทันที

ข้อ	ประเภทชุดทดสอบ	ข้อมูลนำเข้า (Input Arguments)	ผลลัพธ์ที่คาดหวัง (Expected Output)
#1	Visible Test Case	`[["Python", "python", "Data"]]`	`{"python":2,"data":1}`
#2	Visible Test Case	`[["AI", "ML", "ai"]]`	`{"ai":2,"ml":1}`
#3	🔒 Hidden Test Case	`[[]]`	`{}`

ชุดข้อสอบที่ 13 | SmartCareer Core | สอบทฤษฎี (ความรู้ประกอบ)

⏱ เวลา 20 นาที • เกณฑ์ผ่าน 70% • ทักษะ: **JavaScript (FRONTEND)**

Web & Cloud Engineering Fundamentals (Theory)

ข้อสอบทฤษฎีสำหรับทบทวนความรู้พื้นฐานด้าน HTTP, JavaScript และ Cloud (ผลสอบทฤษฎีใช้เพื่อการเรียนรู้ ไม่นำไปคำนวณคะแนนทักษะปฏิบัติส่วนกลาง)

ชุดที่ 13 • ข้อที่ 1/2 JavaScript Block Scope

EASY | 50 คะแนน | ปรนัย (Multiple Choice)

📋 **คำถามปรนัย (Question Prompt):**

คำสั่งใดในภาษา JavaScript ที่ใช้ประกาศตัวแปรแบบ Block Scope และสามารถเปลี่ยนค่าใหม่ได้?

🛠 **ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)**

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: let** — ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: var
- ○ ตัวเลือกที่ 3: const
- ○ ตัวเลือกที่ 4: function

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- เมื่อคลิกเลือกตัวเลือก **“let”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **50 คะแนนเต็ม**

ชุดที่ 13 • ข้อที่ 2/2 HTTP Status Code for Conflict

EASY | 50 คะแนน | ปรนัย (Multiple Choice)

📋 **คำถามปรนัย (Question Prompt):**

เมื่อเกิดข้อขัดแย้งของสถานะข้อมูล เช่น การส่งใบสมัครซ้ำหรือ revision ไม่ตรงกัน ควรใช้ HTTP Status Code ใด?

🛠 **ขั้นตอนการกดเลือกคำตอบ (Step-by-Step Actions)**

ในหน้าห้องสอบทฤษฎี ให้คลิกเลือกตัวเลือกที่มีเครื่องหมาย ✓ **คำตอบที่ถูกต้อง** ด้านล่างนี้:

- ◉ **ตัวเลือกที่ 1: 409 Conflict** — ✓ คำตอบที่ถูกต้อง
- ○ ตัวเลือกที่ 2: 401 Unauthorized
- ○ ตัวเลือกที่ 3: 404 Not Found
- ○ ตัวเลือกที่ 4: 502 Bad Gateway

✅ **ผลลัพธ์ที่ต้องแสดงบนหน้าจอ (Expected Results)**

- เมื่อคลิกเลือกตัวเลือก **“409 Conflict”** ปุ่มวงกลม (Radio Button) ของตัวเลือกนั้นจะถูกเลือกและสามารถกด **“บันทึกฉบับร่าง (Autosave)”** เพื่อเก็บสถานะไว้ได้
- เมื่อเลือกครบทุกข้อและกดปุ่ม **“ยืนยันส่งข้อสอบทฤษฎี”** ด้านบนขวา ระบบจะให้คะแนนข้อนี้ **50 คะแนนเต็ม**
- **ผลลัพธ์รวมของชุดข้อสอบทฤษฎี**: แสดงหน้าสรุปผล **คะแนนสุทธิ (Final Score): 100% (ผ่านเกณฑ์มาตรฐาน ≥70%)** บันทึกลงในตารางประวัติการเข้าสอบ โดยไม่ไปแก้ไขคะแนนทักษะปฏิบัติบนกราฟเรดาร์